

Objektorientierung

Inhalt

- Klassen und Objekte
- Daten und Funktionen
- Vererbung
- Spezielle Vererbungstypen
- Spezielle Klassen

Objektorientierung

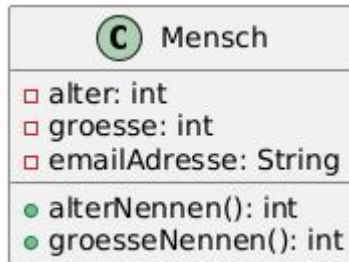
- In der Objektorientierung werden Sachverhalte durch Objekte und ihre Kommunikation abgebildet
- Wesentliche Komponenten: Klassen, Objekte, Messages
- In den nächsten Vorlesungen werden wir den Umgang mit diesen drei Komponenten erlernen.
- Zeitgleich erlernen wir, Klassenrelationen als Diagramme darzustellen, und korrekt zu implementieren

Klassen und Objekte

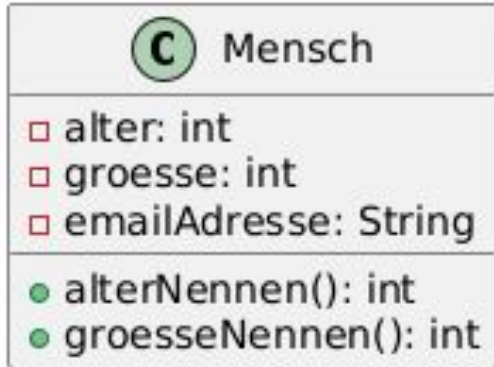
- Eine Klasse beschreibt einen Bauplan für ein Objekt
- Sie legt fest, welche Daten und Funktionen Objekte haben
- Ein Objekt ist eine konkrete Instanz einer Klasse

Beispiel: Ein Mensch besitzt ein Alter, eine Größe, und eine E-mail-Adresse. Ein Mensch kann sein Alter und seine Größe nennen. Sebastian ist 37, 185cm groß, und wenn er gefragt wird, nennt er sein Alter und seine Größe.

Beispiel Klasse Mensch

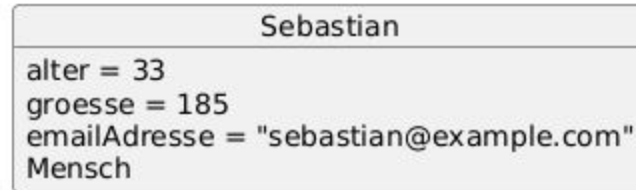
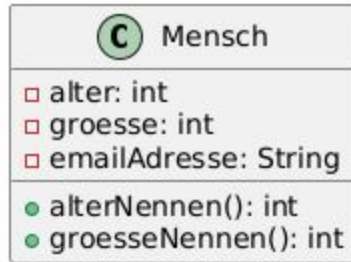


Beispiel Klasse Mensch: PlantUML



```
@startuml
class Mensch {
    - alter: int
    - groesse: int
    - emailAdresse: String
    + alterNennen(): int
    + groesseNennen(): int
}
@enduml
```

Beispiel Klasse Mensch + Instanz



In python

```
class Mensch:
    alter = 0
    groesse = 0
    email_adresse = ""

    def __init__(self, alter, groesse, email) -> None:
        self.alter = alter
        self.groesse = groesse
        self.email_adresse = email

sebastian = Mensch(33, 185, 'sebastian.gutsche@th-owl.de')
```


Daten und Funktionen

- Eine Klasse hat zwei verschiedene Komponenten: Daten und Funktionen
- Daten legen fest, welche Daten eine konkrete Instanz besitzt.
- Funktionen legen fest, wie sich eine konkrete Instanz verhält.
- Grundsätzlich zwei verschiedene Arten von Funktionen: Klassen- und Memberfunktionen (Methoden).
- Klassenfunktionen greifen nicht auf die Daten eines Objekts zu, Memberfunktionen schon.

Ein Mensch besitzt ein Alter, eine Größe, und eine Email-Adresse. Ein Mensch kann sein Alter und seine Größe nennen.

In python

```
class Mensch:
    alter = 0
    groesse = 0
    email_adresse = ""

    def __init__(self, alter, groesse, email) -> None:
        self.alter = alter
        self.groesse = groesse
        self.email_adresse = email

    def reden(self):
        return f'Ich bin {self.alter} Jahre alt und {self.groesse} cm groß'

sebastian = Mensch(33, 185, 'sebastian.gutsche@th-owl.de')

sebastian.reden()
'Ich bin 33 Jahre alt und 185 cm groß'
```

Klassenfunktionen

- Klassenfunktionen nutzen keine Referenz auf das Objekt selbst
- Man kann sie direkt auf der Klasse aufrufen, aber auch auf Objekten

In python

```
class Mensch:
    alter = 0
    groesse = 0
    email_adresse = ""

    def __init__(self, alter, groesse, email) -> None:
        self.alter = alter
        self.groesse = groesse
        self.email_adresse = email

    def reden(self):
        return f'Ich bin {self.alter} Jahre alt und {self.groesse} cm groß'

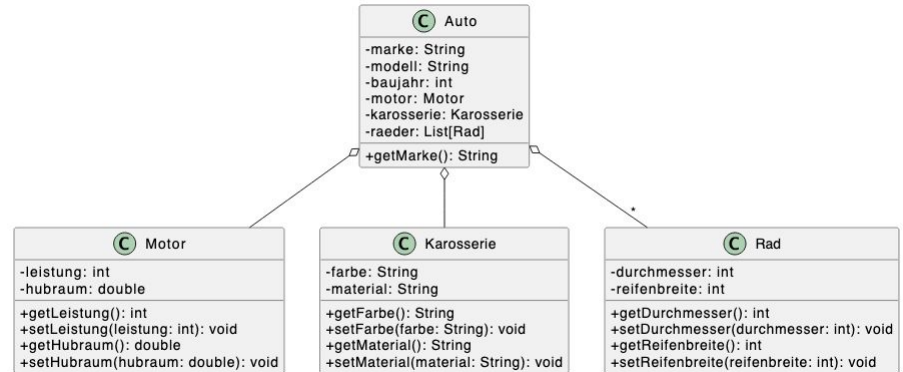
    @classmethod
    def gruesse(cls):
        return 'Hallo'

sebastian = Mensch(33, 185, 'sebastian.gutsche@th-owl.de')

sebastian.reden()
'Ich bin 33 Jahre alt und 185 cm groß'
sebastian.gruesse()
'Hallo'
Mensch.gruesse()
'Hallo'
```

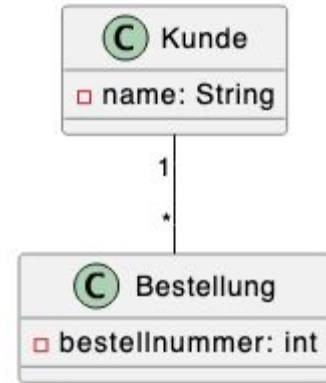
Wie verhalten sich Klassen zueinander: Aggregation

- Aggregation bedeutet, dass eine Klasse aus mehreren Klassen zusammengesetzt ist
- Die aggregierten Klassen können auch einzeln existieren, jedoch kann die Klasse, die aggregiert, nicht ohne ihre Teile existieren



Schwächeres Verhältnis: Assoziation

- Assoziation bedeutet, es gibt eine Beziehung zwischen zwei Klassen
- Zum Beispiel kann einem Kunden eine Bestellung assoziiert sein
- Während bei der Aggregation die aggregierende Klasse nicht ohne ihre Teile existieren kann, kann bei der Assoziation jede Klasse unabhängig existieren. Die Assoziation erfolgt normalerweise über eine zusätzliche Struktur



Vererbung

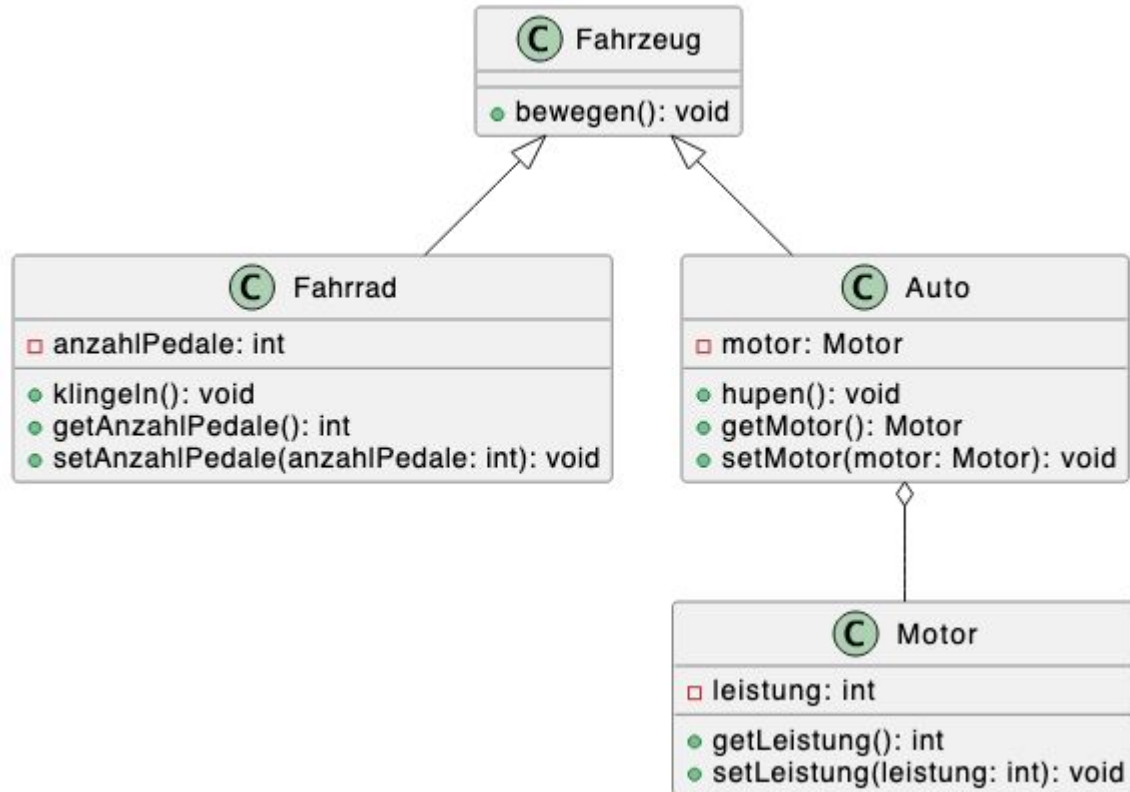
- Eine der wichtigsten Funktionen von Klassen ist die Vererbung
- Bei der Vererbung wird eine “speziellere” Klasse von einer “allgemeineren” abgeleitet
- Funktionen und Daten können weiter verwendet oder überschrieben werden.

Beispiel Fahrzeug

- Ein Fahrzeug kann sich bewegen.
- Ein Fahrrad ist ein Fahrzeug mit zwei Pedalen. Es kann klingeln.
- Ein Auto ist ein Fahrzeug mit einem Motor. Es kann hupen.

In einem Objektorientierten Modell können Fahrrad und Auto die Bewegungseigenschaft von Fahrzeug erben.

Diagram



In python

```
class Fahrzeug:
    def bewegen(self):
        return 'brum'

class Auto(Fahrzeug):
    motor = 1

    def hupen(self):
        return 'hup'

class Fahrrad(Fahrzeug):
    pedale = 2

    def klingeln(self):
        return 'ring'

fahrrad = Fahrrad()
fahrrad.bewegen()
'brum'
fahrrad.pedale
2
fahrrad.hupen()
# Fehler
```

Wofür dient Vererbung

- Strukturierung
- Einheitliche Interfaces
- Vermeidung von Codeduplikation

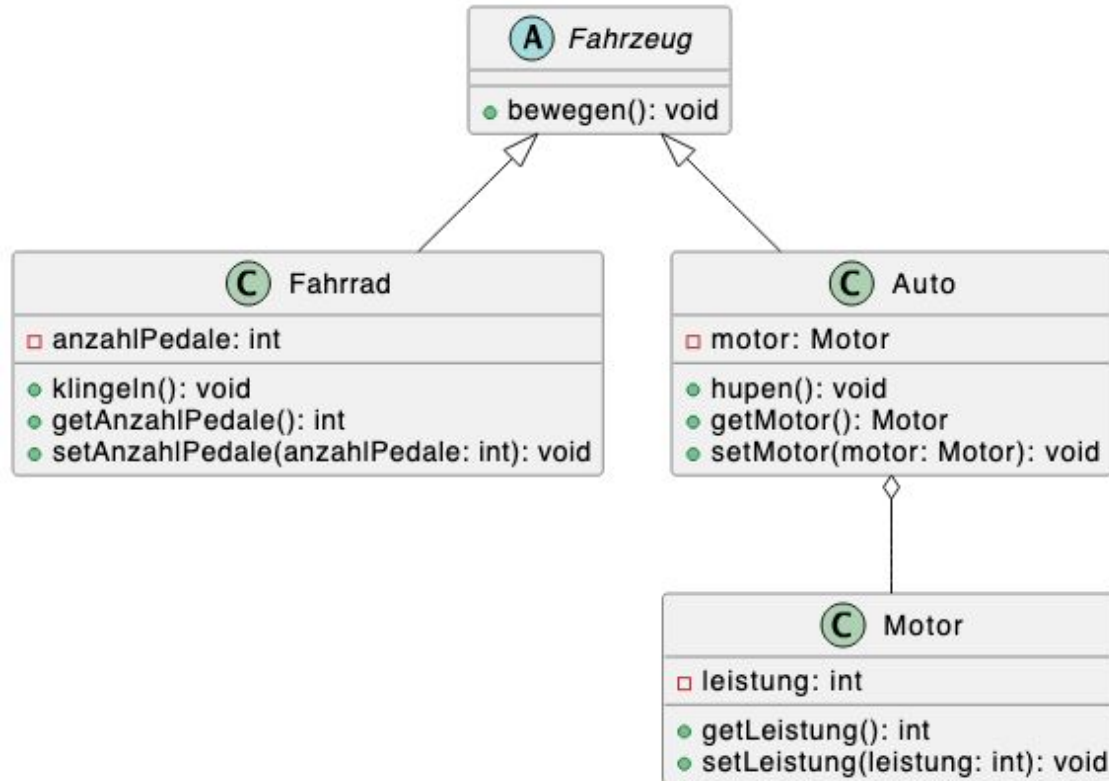
Verallgemeinerung und Spezialisierung treten natürlich auf: Merkt man, dass bestimmte Daten oder Funktionen in verschiedenen Klassen benötigt werden, so kann man dies durch Vererbung beschreiben.

Aber: Vererbung sollte zur Programmlogik passen.

Abstrakte und konkrete Klassen

- Abstrakte Klassen dienen nicht dazu, dass man Instanzen von ihnen erzeugt
- Lediglich Vererbung ist möglich
- Dies ermöglicht es, ein allgemeines Interface zu definieren, welches uns erlaubt, Funktionen mit verschiedenen Subtypen (abgeleitete Klassen) interagieren zu lassen
- In unserem Beispiel kann Fahrzeug abstrakt erzeugt werden

Abstrakte und konkrete Klassen Diagramm



In python

```
class Fahrzeug:
    def __init__(self):
        raise NotImplementedError

    def bewegen(self):
        return 'brum'

class Auto(Fahrzeug):
    motor = 1

    def hupen(self):
        return 'hup'

class Fahrrad(Fahrzeug):
    pedale = 2

    def klingeln(self):
        return 'ring'

fahrzeug = Fahrzeug() # Fehler
```

Möglichkeiten der Vererbung

- Es gibt niemals nur eine Möglichkeit, ein System zu designen
- Es sollten Verschiedene Möglichkeiten angegeben werden, und diese sollten immer mit Vor- und Nachteilen diskutiert werden.
- Dinge, die bei der Diskussion zu beachten sind:
 - Ist das System (eindeutig) erweiterbar?
 - Ist das System übersichtlich?
 - Ist das System frei von Uneindeutigkeit?

Beispiel Kunde

Angenommen, wir besitzen ein Geschäft. Wir haben verschiedene Arten von Kunden

- Registrierte Kunden
- Kunden mit Bonuskarte
- Premiumkunde (darf auf Rechnung kaufen)

Wir wollen im folgenden verschiedene Arten der objektorientierten Modellierung besprechen, inklusive Vor- und Nachteile. Ein gutes Design enthält an dieser Stelle immer mehrere Vorschläge.

Analyse der Objekte

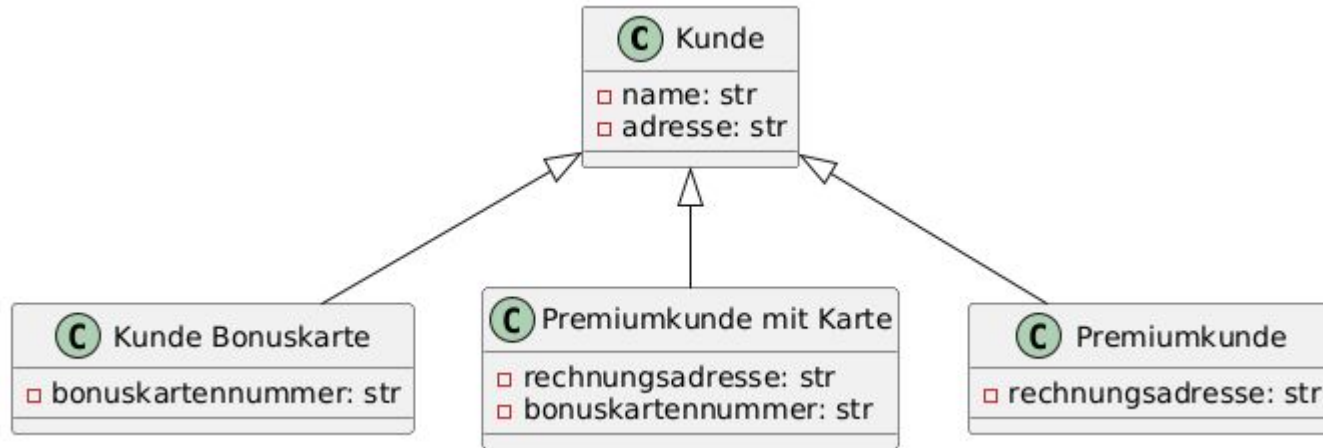
In diesem Beispiel sind 4 verschiedene Kundentypen vorhanden:

- Ein (registrierter) Kunde
- Ein Kunde mit Bonuskarte
- Ein Premiumkunde
- Ein Premiumkunde mit Bonuskarte

Diese 4 verschiedenen Fälle muss unsere Struktur abbilden können.

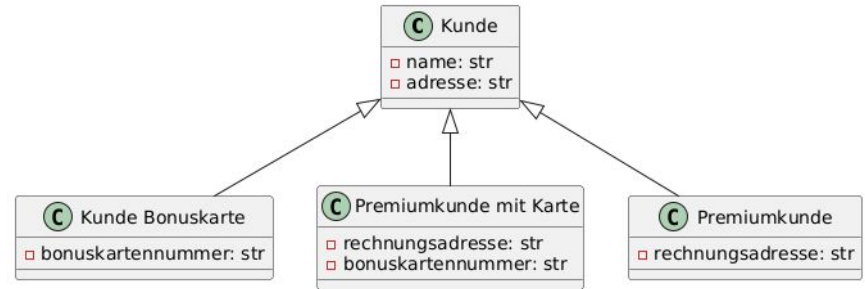
Einfache Vererbung

Wir erzeugen eine Klasse Kunde, von der unsere drei Spezialisierungen abgeleitet werden



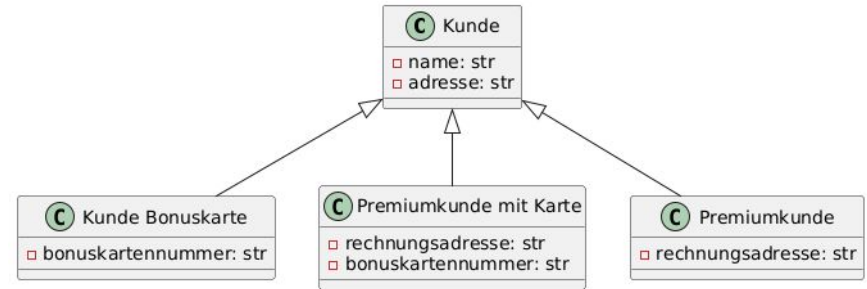
Vor- und Nachteile

- Wir erzeugen eine klare Hierarchie, jede Ableitung ist erwartbar, aber
- wir duplizieren die Dateneinträge für Rechnungsadresse und Bonuskartennummer
- Wir können zwar weitere Typen von Kunden hinzufügen, werden aber evtl. weitere Daten duplizieren

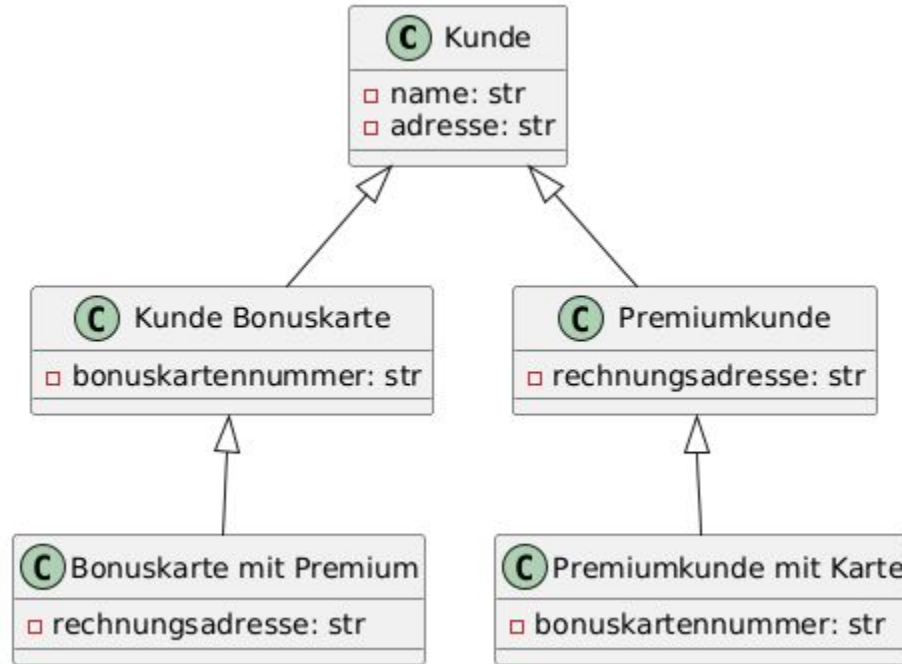


Unsere Fragen von vorher

- Ist das System (eindeutig) erweiterbar?
 - Vermutlich ja
- Ist das System übersichtlich?
 - Eine Hierarchieebene, also vermutlich schon
- Ist das System frei von Uneindeutigkeit?
 - Die Klassen sind eindeutig, jedoch nicht die Daten

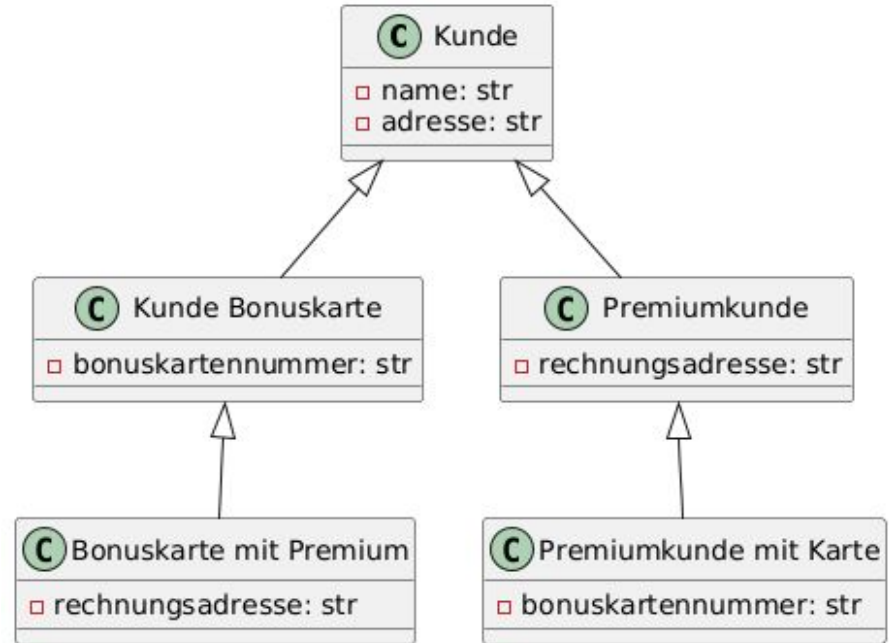


Absolutes Negativbeispiel: Verschiedene Ableitungen



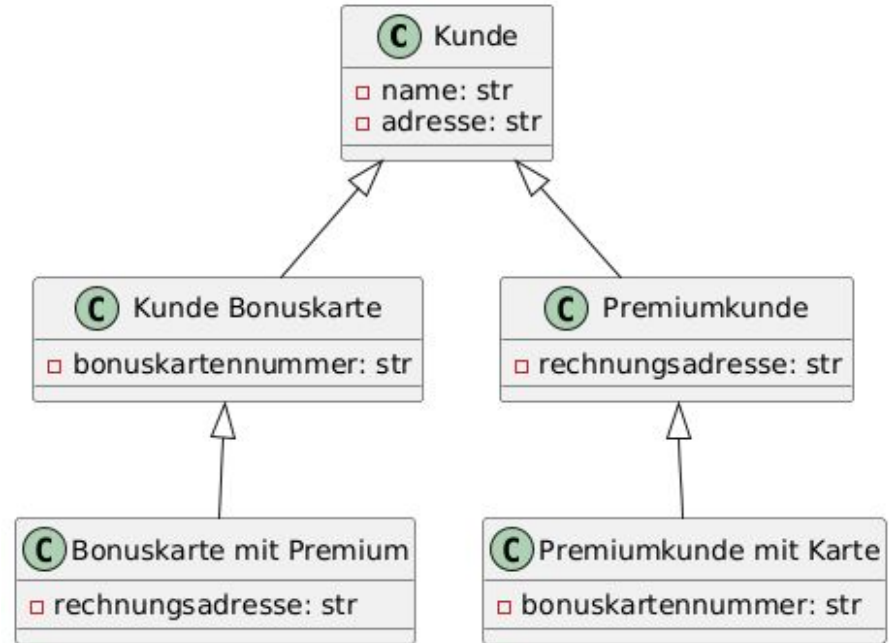
Nachteile

Wir sehen dass die unteren beiden Klassen faktisch identisch sind (in Code und in Problemanalyse). Dies sollte immer vermieden werden.

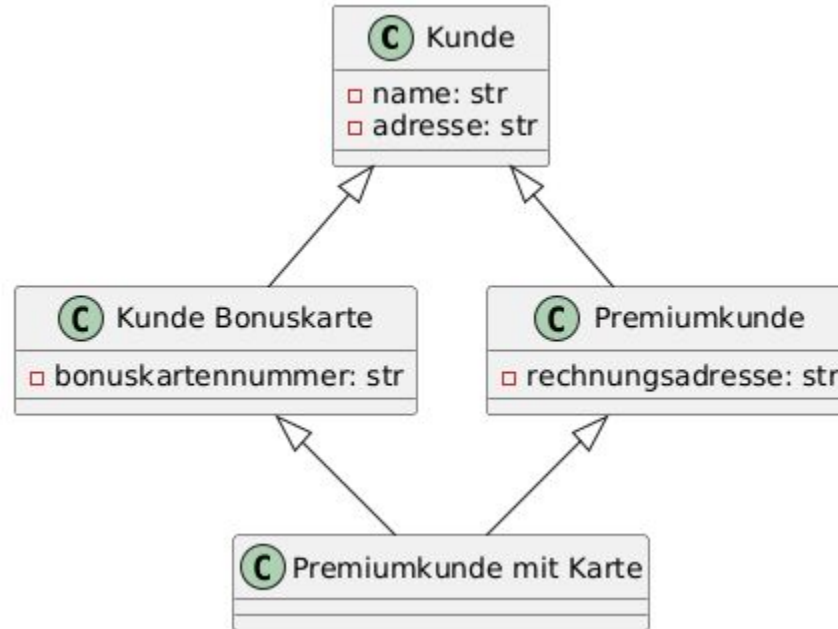


Unsere Fragen von vorher

- Ist das System (eindeutig) erweiterbar?
 - Eher nein, siehe unten.
- Ist das System übersichtlich?
 - Nein, es ist uneindeutig, und besitzt unnötige Klassen.
- Ist das System frei von Uneindeutigkeit?
 - Nein, die beiden speziellsten Klassen sind identisch in ihrer Anwendung

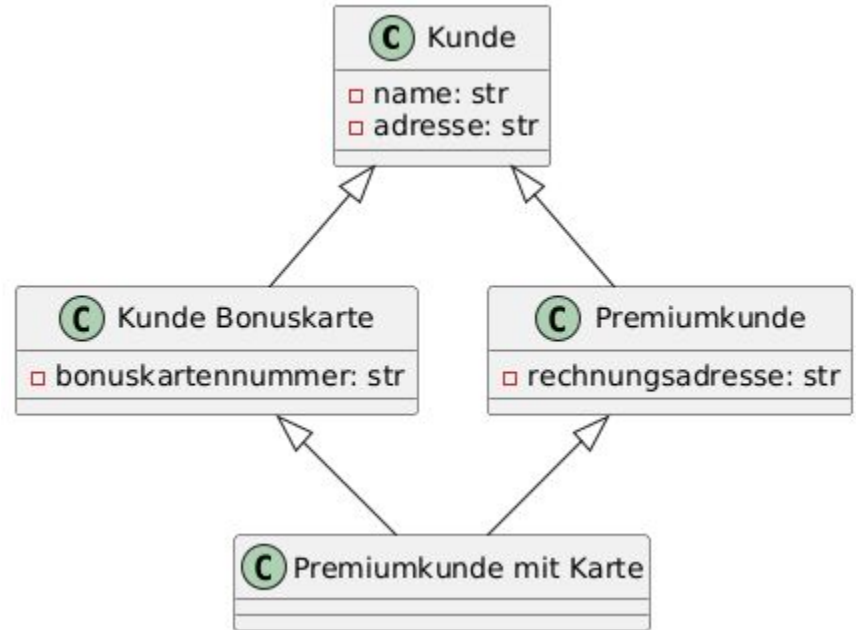


Multiple Vererbung



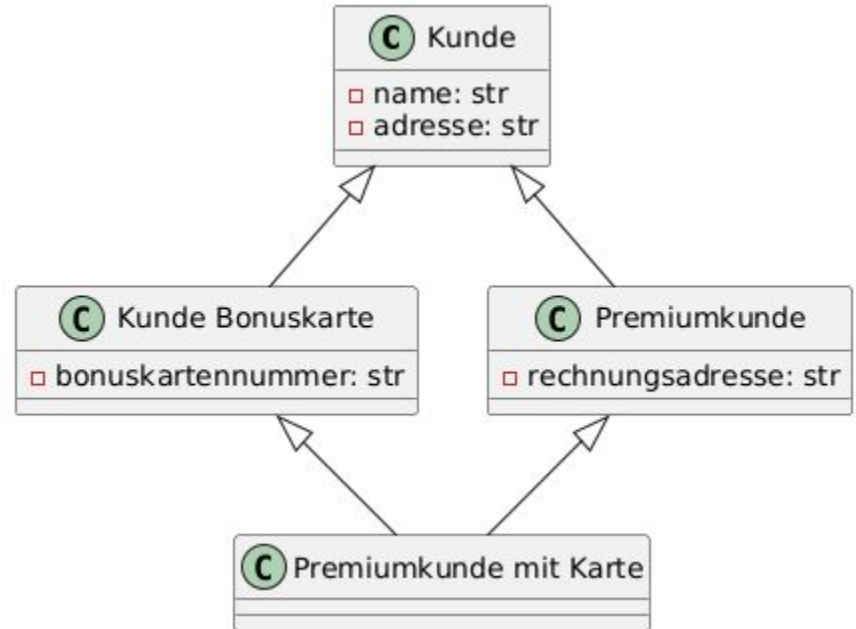
Vor- und Nachteile

- Keine Duplikation der Daten
- Multiple Vererbung kann schnell zu unübersichtlichem Code führen. Es muss beachtet werden, wie Method resolution etc. funktioniert (siehe Python Beispiel)



Unsere Fragen von Vorher

- Ist das System (eindeutig) erweiterbar?
 - Eher nein. Wo ist der eindeutige Anknüpfungspunkt
- Ist das System übersichtlich?
 - Ja
- Ist das System frei von Uneindeutigkeit?
 - Tatsächlich ja, jedes Datum wurde nur einmal definiert.



Multiple Inheritance in python

```
class Kunde:
    def __init__(self, name, adresse):
        self.name = name
        self.adresse = adresse

class BonuskartenKunde(Kunde):
    def __init__(self, name, adresse, bonuskarte):
        super().__init__(name, adresse)
        self.bonuskarte = bonuskarte

class PremiumKunde(Kunde):
    def __init__(self, name, adresse, rechnungsadresse):
        super().__init__(name, adresse)
        self.rechnungsadresse = rechnungsadresse

class BonusPremiumKunde(BonuskartenKunde, PremiumKunde):
    def __init__(self, name, adresse, bonuskarte, rechnungsadresse):
        self.name = name
        self.adresse = adresse
        self.bonuskarte = bonuskarte
        self.rechnungsadresse = rechnungsadresse
```

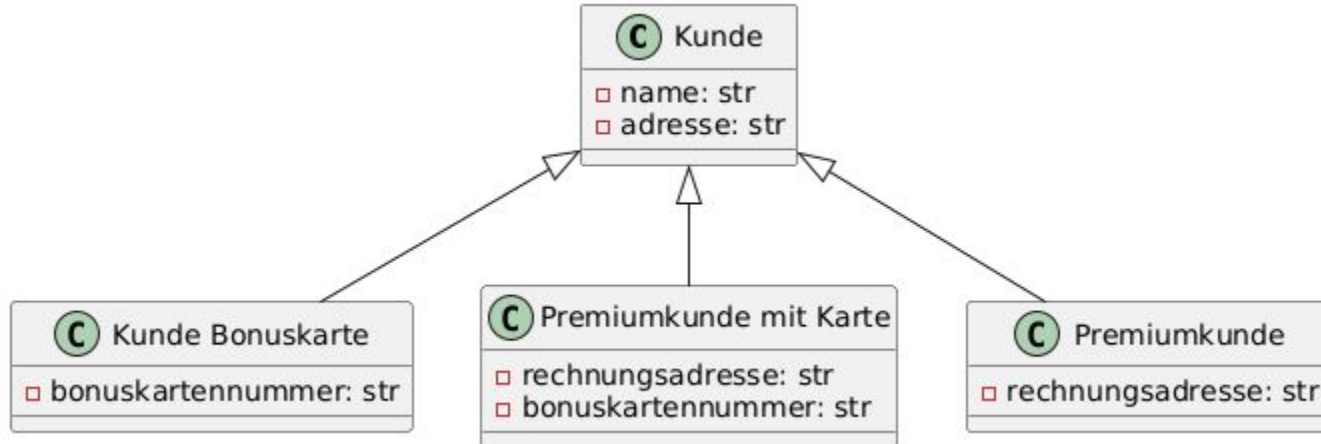
Multiple inheritance

- An dieser Stelle sorgt multiple inheritance für ein angenehmes Diagram, aber eigentlich unangenehmen code.
- Sie sollte daher vermieden werden.
- Später lernen wir ein anderes Beispiel für multiple inheritance kennen: Mix-ins.

Data inheritance

- Wir haben bisher nur data inheritance gesehen
- Diese ist üblich, führt aber in der Praxis häufig zu Verwirrung
- Wie können wir dies beseitigen?

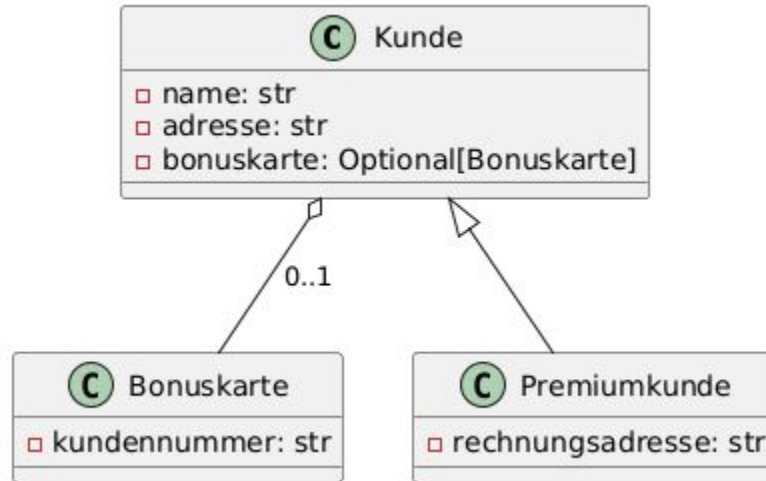
In welcher Klasse wurde nochmal Adresse definiert?



Möglichkeiten data inheritance aufzulösen

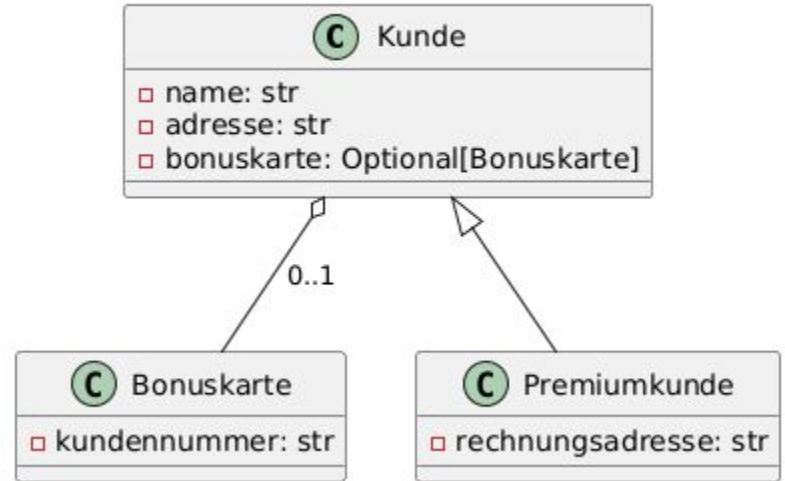
- Data inheritance kann durch weitere Objekte aufgelöst werden
- Schauen wir noch einmal:
 - Registrierte Kunden
 - Kunden mit Bonuskarte
 - Premiumkunde (darf auf Rechnung kaufen)
- Die Bonuskarte ist ein Objekt, welches ein Kunde haben kann

Bonuskarte als Objekt



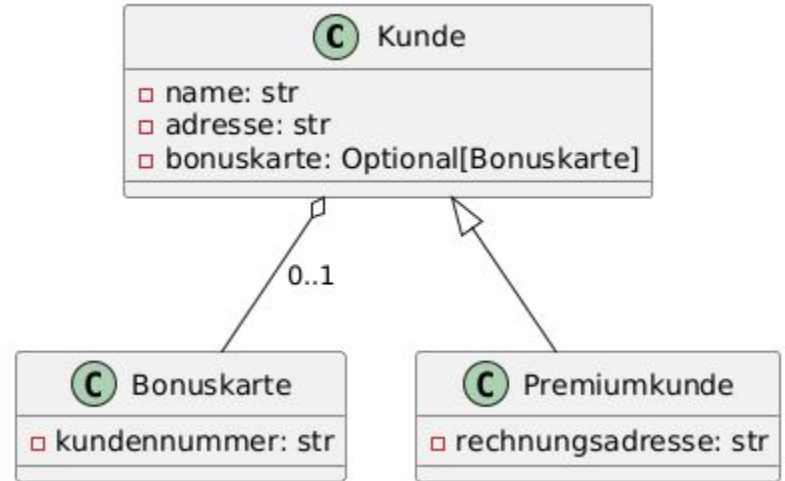
Vor- und Nachteile

- Die Bonuskarte steht nun außerhalb.
- Als Objekt kann sie weitere Daten speichern, wie zum Beispiel Bonuspunkte
- Wir haben jedoch noch weitere Datenvererbung.

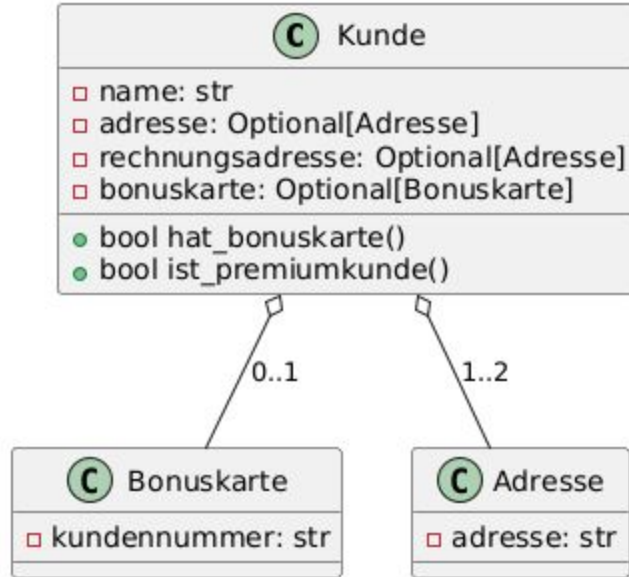


Unsere Fragen von Vorher

- Ist das System (eindeutig) erweiterbar?
 - Funktionen der Bonuskarte wären gut erweiterbar
- Ist das System übersichtlich?
 - Ja
- Ist das System frei von Uneindeutigkeit?
 - Nicht unbedingt. Warum ist Bonuskarte ein Objekt, aber nicht die Eigenschaft Premium

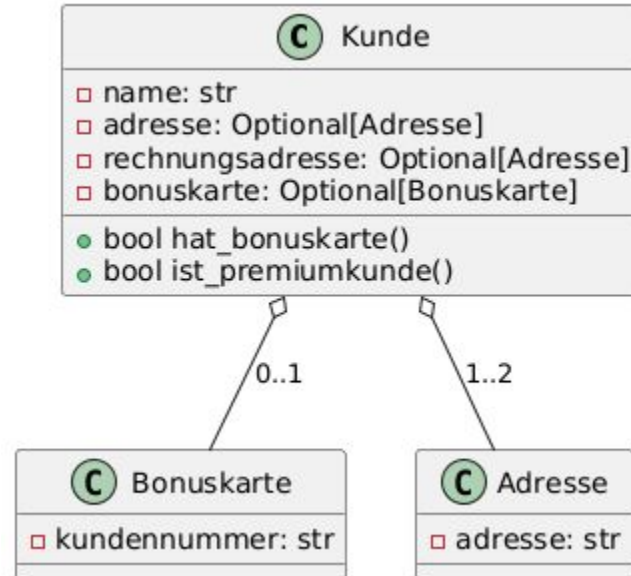


Alles als Objekte



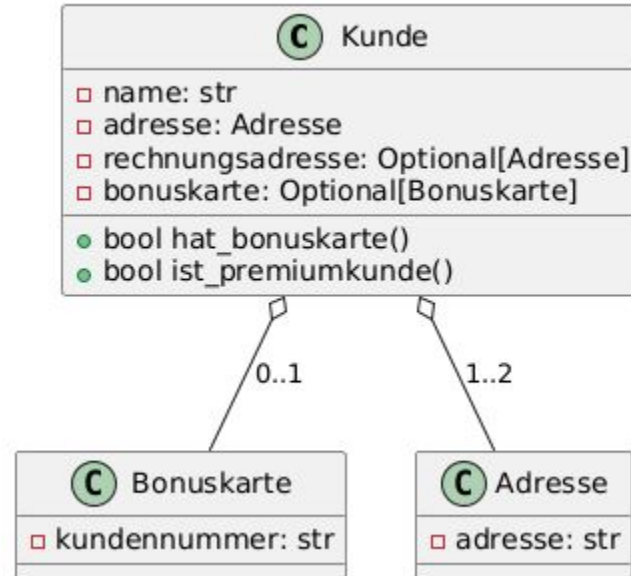
Vor- und Nachteile

- Alle Objekte stehen nun Außerhalb, und die Funktionen der Klasse sind nun alle in der Klasse Kunde vorhanden
- Aber: Wir haben keine Klassen mehr für andere Kunden

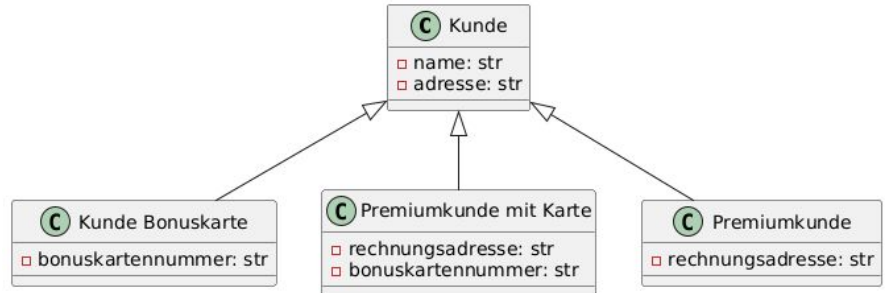
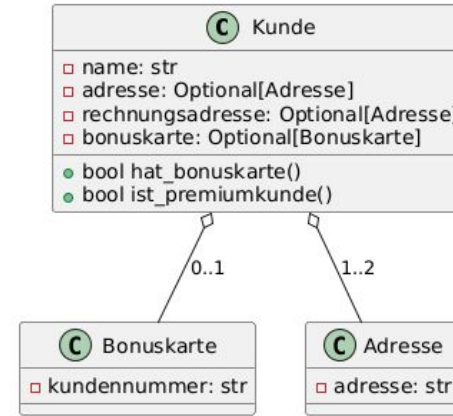
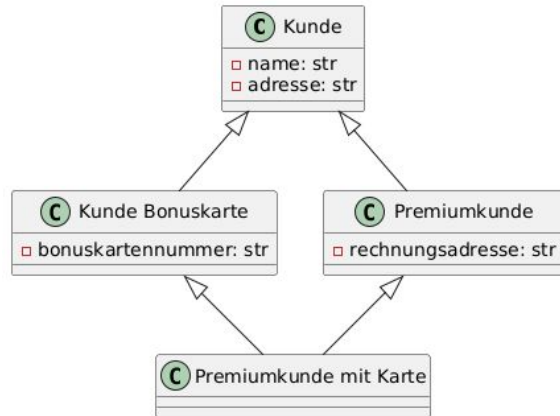
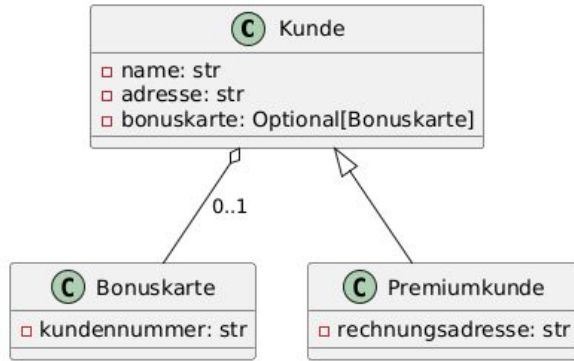


Unsere Fragen von Vorher

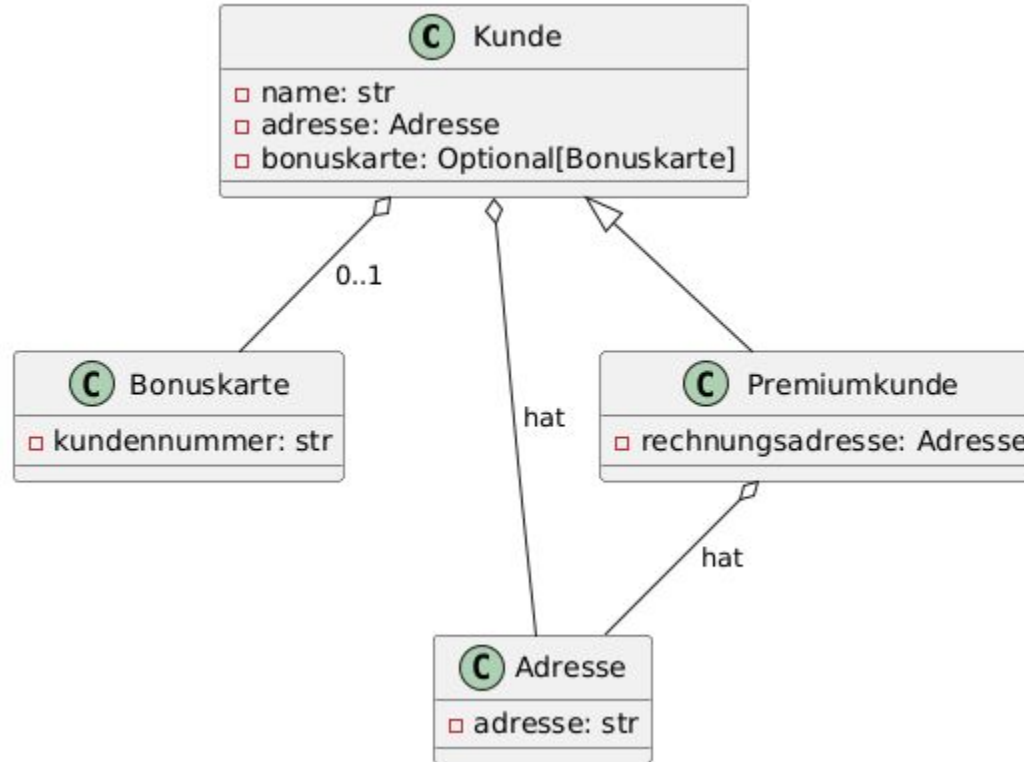
- Ist das System (eindeutig) erweiterbar?
 - Ja. Da wir keine Vererbung haben, wissen wir, wo wir erweitern müssen
- Ist das System übersichtlich?
 - Ja
- Ist das System frei von Uneindeutigkeit?
 - Nicht unbedingt. Wir können nicht auf Grund des Typs entscheiden, welche Operation auf einem Objekt ausgeführt werden soll



Was ist nun das beste?

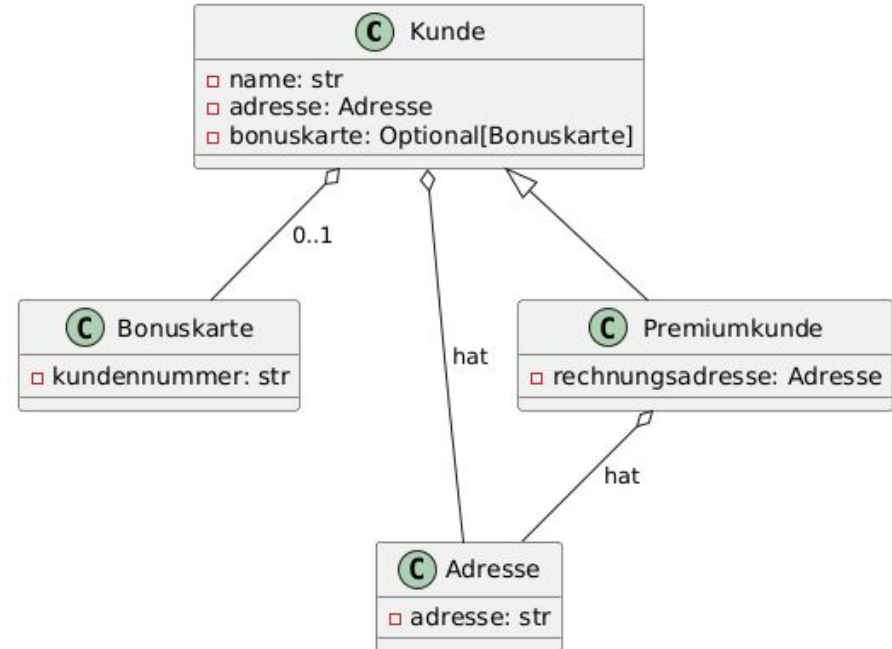


Meine Antwort (Opinionated)



Warum?

- Ist das System (eindeutig) erweiterbar?
 - Ja. Wir können neue Kundentypen oder neue Kundeneigenschaften einfach an einer eindeutigen Stelle einsetzen.
- Ist das System übersichtlich?
- Ja
- Ist das System frei von Uneindeutigkeit?
 - Ja. Nichts wird doppelt definiert. Adresstyp wird weiterverwendet.



In python

```
class Bonuskarte:
    def __init__(self, bonusnummer):
        self.bonusnummer = bonusnummer

    def has_number(self):
        if self.bonusnummer:
            return True
        return False

class Adresse:
    def __init__(self, adresse):
        self.adresse = adresse

class Kunde:
    def __init__(self, adresse, bonuskarte):
        self.adresse = Adresse(adresse)
        self.bonuskarte = Bonuskarte(bonuskarte)

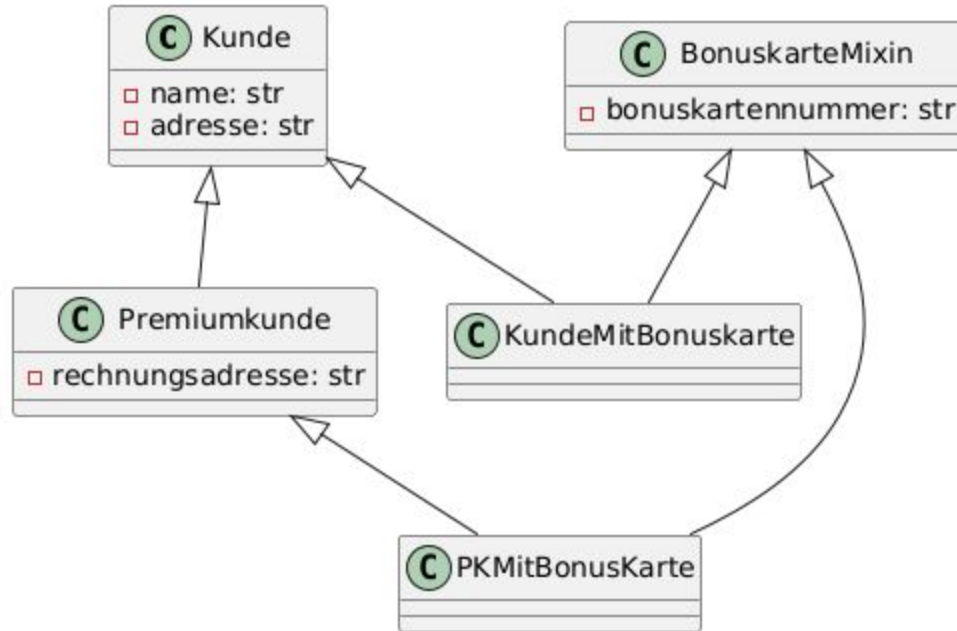
    def has_bonuskarte(self):
        return self.bonuskarte.has_number()

class Premiumkunde(Kunde):
    def __init__(self, adresse, bonuskarte, rechnungsadresse):
        super().__init__(adresse, bonuskarte)
        self.rechnungsadresse = Adresse(rechnungsadresse)
```


Spezielle Vererbung: Mixins

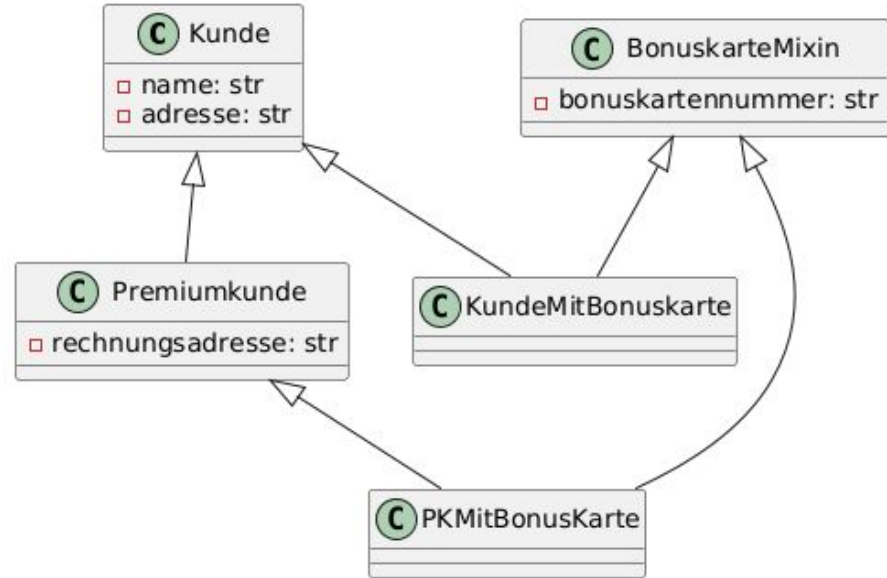
- Mixins sind Klassen, die anderen Klassen bestimmte Funktionen hinzufügen
- Beispiele sind:
 - Hashing
 - Validation
 - Caching
- Wir geben zuerst ein Beispiel mit Kundenkarte

Bonuskartenmixin



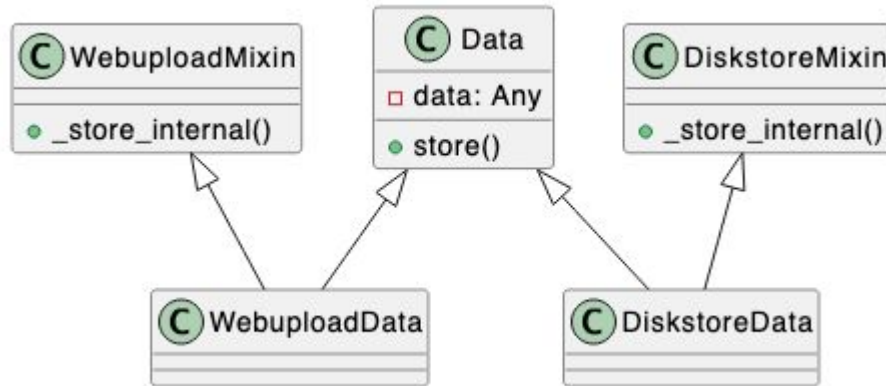
Vor- und Nachteile

- Wir haben nun den Fall von vorher, allerdings ohne Datenduplikationen
- Generell würde man dies jedoch als schlechten Stil betrachten, denn Mixins sollten keine Daten vererben, nur Funktionalität



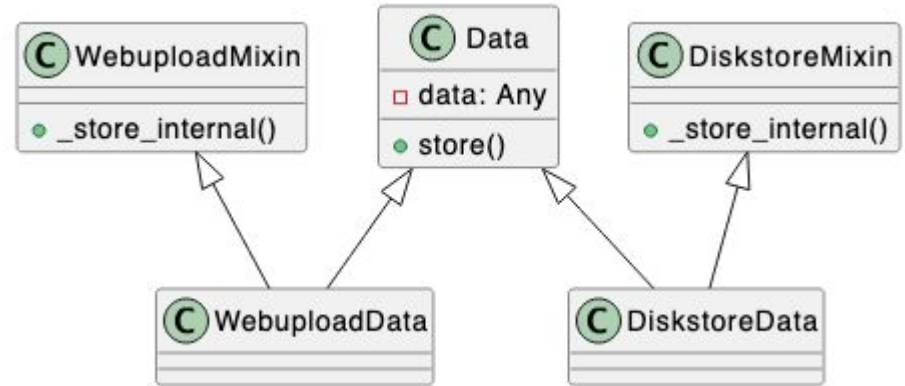
Ein realistisches Beispiel

Wir betrachten eine Klasse, die sowohl zum Webupload als auch zum Speichern auf die Festplatte benutzt werden kann.



Übersicht

- store in der Data Klasse ruft _store_internal auf.
- _store_internal wird von den Mixin Klassen definiert
- Je nach Mixin entsteht ein anderer Upload

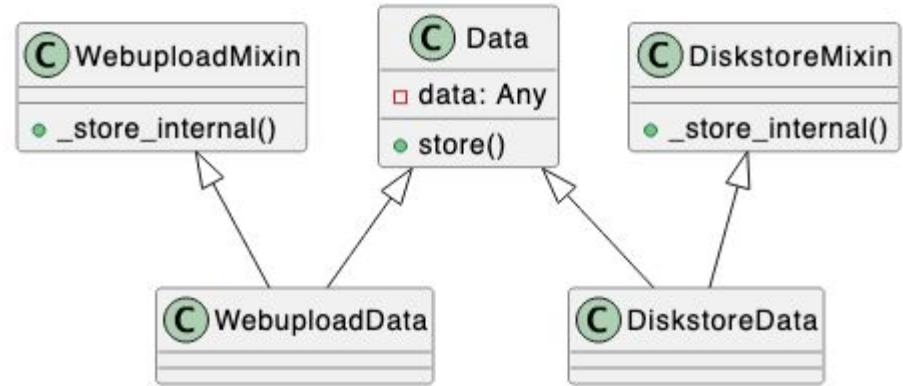


In python

```
1 class Data:
2     def __init__(self, data):
3         self.data = data
4     def store(self):
5         self._store_internal(self.data)
6
7
8 class WebuploadMixin:
9     @staticmethod
10    def _store_internal(data):
11        print(f'Uploading {data} to web')
12
13
14 class DiskstoreMixin:
15     @staticmethod
16    def _store_internal(data):
17        print(f'Storing {data} to disk')
18
19
20 class WebuploadData(Data, WebuploadMixin):
21     pass
22
23
24 class DiskStoreData(Data, DiskstoreMixin):
25     pass
```

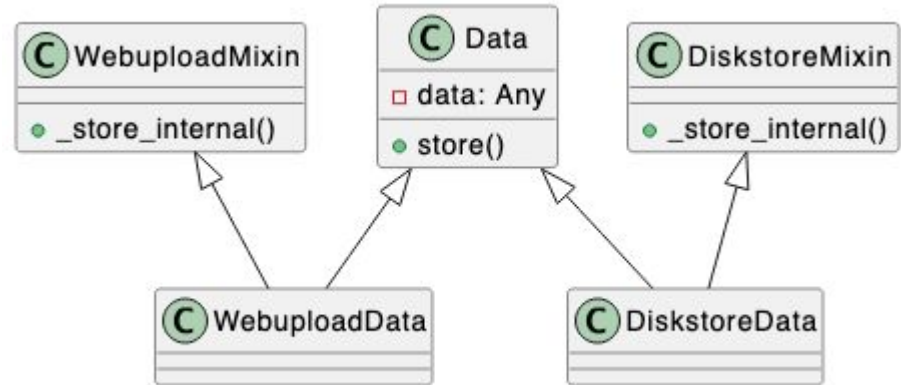
Vor- und Nachteile

- Wir haben keine Codeduplikation
- Wir können einfach um andere Storage modes oder andere Data classes erweitern
- Aber der Flow des codes ist nicht unbedingt klar: `_store_internal` wird ohne Kontext aufgerufen. Hier helfen jedoch Kommentare



Unsere Fragen

- Ist das System (eindeutig) erweiterbar?
 - Ja
- Ist das System übersichtlich?
 - Ja
- Ist das System frei von Uneindeutigkeit?
 - Fast. Eigentlich gibt es keine Uneindeutigkeit, doch die Reihenfolge der Vererbung ist aus dem Diagramm nicht ersichtlich.



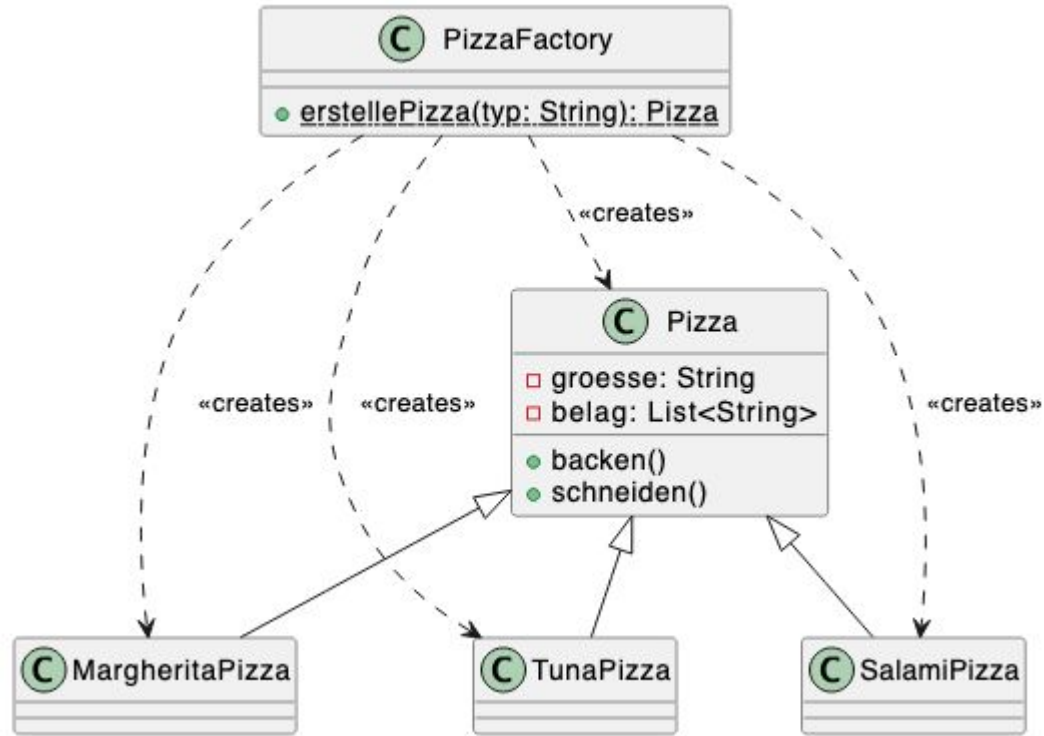
Factory classes

- In der Objektorientierten Programmierung werden oft sogenannte Factories verwendet (als Klassen oder als Objekte).
- Factories sind Klassen, die der Erzeugung eines anderen Objektes dienen. Sie selbst enthalten nur die Eigenschaften, die zur Erzeugung des Produktes benötigt werden.
- Manchmal müssen nicht einmal konkrete Instanzen erzeugt werden.

Beispiel: PizzaFactory

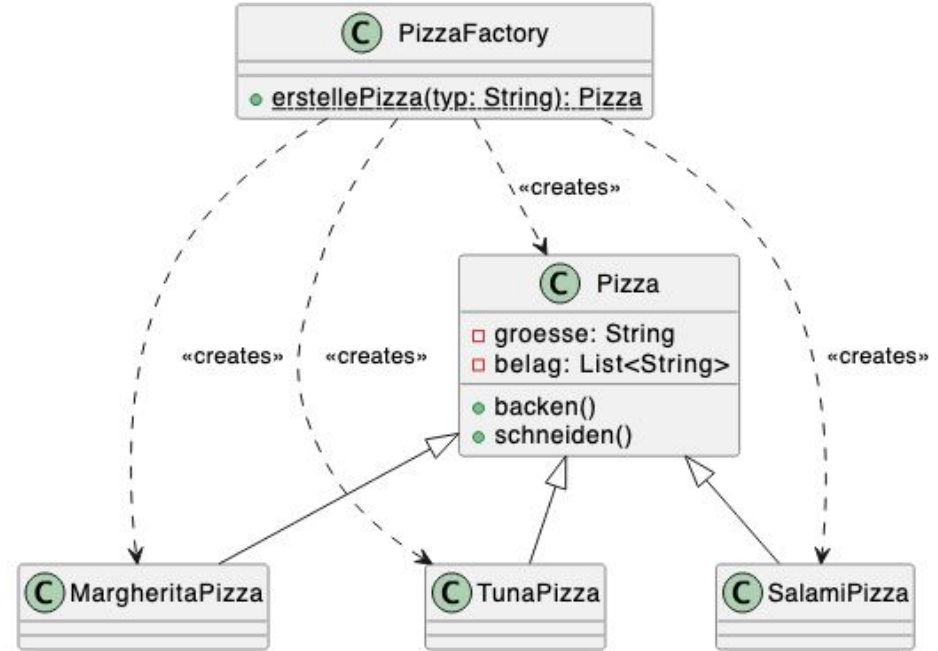
Eine PizzaFactory kann jeweils eine Pizza erzeugen. Sie wird über Zutaten konfiguriert. Je nach Factory wird eine andere Art von Pizza erzeugt. Die Pizza wird mit Zutaten ausgestattet. Eine PizzaFactory kann mehrere Pizzen erzeugen.

PizzaFactory



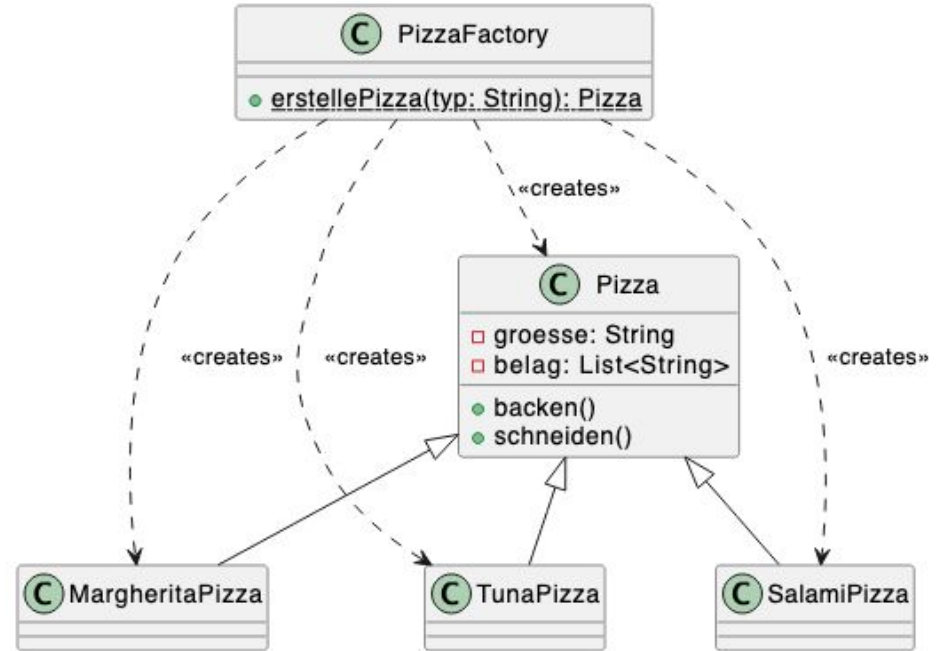
PizzaFactory

- Hier meint der durchgezogene Pfeil, dass Klassen abgeleitet werden.
- Der gestrichelte Pfeil gibt die Erzeugung an



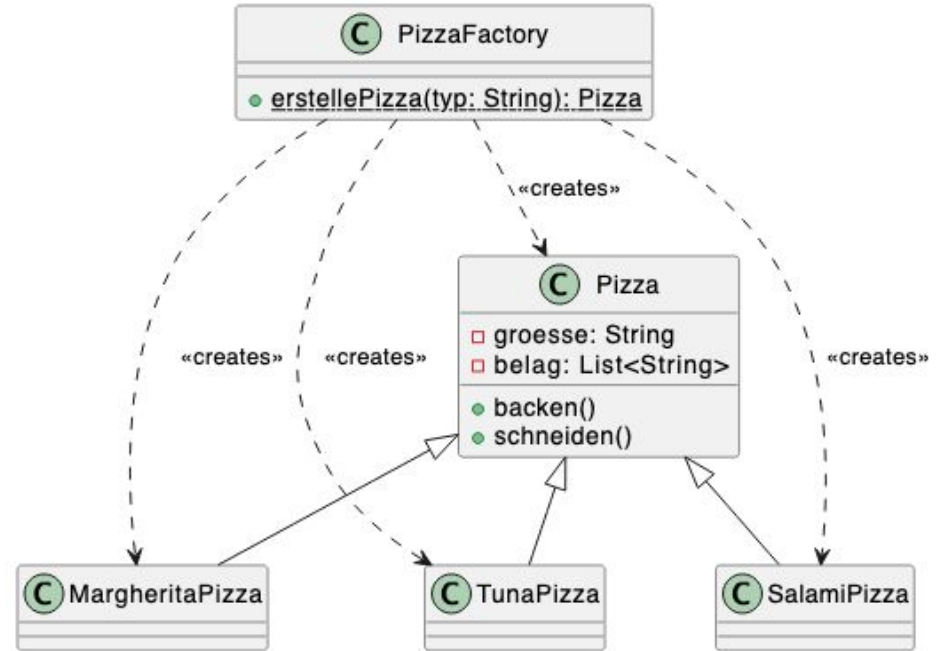
PizzaFactory

- Man kann nun aus der Factory ableiten, was erzeugt werden soll. Alle Factories haben die gleiche Signatur
- Eine Factory kann wiederverwendet werden.
- Besonders wichtig: In stark typisierten Sprachen ist an jeder Stelle der Typ bekannt.



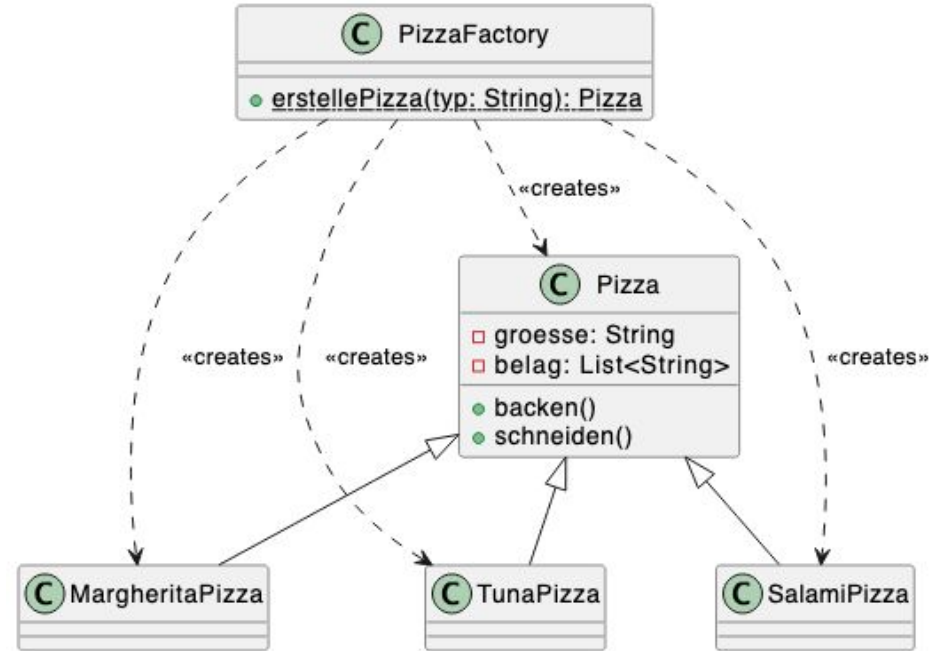
PizzaFactory

- Besonders wichtig: In stark typisierten Sprachen ist an jeder Stelle der Typ bekannt.
 - Dies bedeutet, dass die Factories allgemein übergeben werden können, um ein Objekt von Typ Pizza zu erzeugen.
 - Alle Interfaces müssen nur auf die Basisklassen angepasst werden, niemals auf die konkreten.



Unsere Fragen

- Ist das System (eindeutig) erweiterbar?
 - Ja
- Ist das System übersichtlich?
 - Ja
- Ist das System frei von Uneindeutigkeit?
 - Ja
- Haben wir also etwas optimales entdeckt?
 - Leider nein, denn der Code wird nun leider unübersichtlicher.



In python

```
class Pizza:
    def __init__(self):
        self.groesse = ""
        self.belag = []
    def backen(self):
        pass
    def schneiden(self):
        pass

class MargheritaPizza(Pizza):
    def __init__(self):
        super().__init__()
        self.belag = ["Tomatensoße", "Mozzarella", "Basilikum"]
    def backen(self):
        print("Backe eine Margherita Pizza")
    def schneiden(self):
        print("Schneide die Margherita Pizza in Stücke")

class SalamiPizza(Pizza):
    def __init__(self):
        super().__init__()
        self.belag = ["Tomatensoße", "Mozzarella", "Salami"]
    def backen(self):
        print("Backe eine Salami Pizza")
    def schneiden(self):
        print("Schneide die Salami Pizza in Stücke")
```

```
class TunaPizza(Pizza):
    def __init__(self):
        super().__init__()
        self.belag = ["Tomatensoße", "Mozzarella", "Thunfisch", "Zwiebeln"]
    def backen(self):
        print("Backe eine Thunfisch Pizza")
    def schneiden(self):
        print("Schneide die Thunfisch Pizza in Stücke")

class PizzaFactory:
    @staticmethod
    def erstelle_pizza(typ: str) -> Pizza:
        if typ == "Margherita":
            return MargheritaPizza()
        elif typ == "Salami":
            return SalamiPizza()
        elif typ == "Tuna":
            return TunaPizza()
        else:
            raise ValueError("Unbekannter Pizza-Typ")
```